

Ankyra: Sound Neuro-Symbolic Reasoning with Verifiable Proofs

Vladimir Lapshin*

Abstract

We present Ankyra, a domain-general neuro-symbolic reasoning system built on a strict separation of roles: the language model *proposes* a formalization, and a deterministic symbolic engine *decides* it. The system spans a family of decidable formalisms — from definite Horn clauses and stratified negation to positive first-order logic with disjunction and quantifiers, finite-domain constraints, exact arithmetic, and defeasible defaults with specificity. Each formalism comes with a sound decision procedure and a declared fragment. Every answer carries a machine-checkable proof trace, and no statement enters the theory without either a verbatim quotation or an explicit hypothesis tag. We report a demonstration-scale evaluation on standard collections: on covered slices the system achieves high accuracy while producing no confidently wrong conclusions, and it abstains honestly (`out_of_fragment`) where the implemented logic cannot express the problem. We also compare against two open-source neuro-symbolic systems — Logic-LM and LINC — on identical inputs and the same model, and show that under a small, inexpensive model the binding constraint is the translation interface rather than the reasoning. We make no claim to state-of-the-art performance: the evaluation uses fixed subsamples rather than full test sets, and we state the statistical and methodological limitations explicitly.

1 Introduction

Large language models read text well and translate it into a precise formal language. They do poorly what symbolic procedures do well: they derive consequences *soundly*, track exactly which premises were used, and refuse to guess. A free chain of thought sounds plausible, but it does not distinguish a correct step from a merely convincing one: it carries neither a certificate of soundness nor a history from which one could reconstruct what the answer rests on. Verifiability is therefore the central problem of reasoning with language models.

Ankyra [1] gives each side the work it is good at. The language model reads the text and does two things: it decomposes the problem into *structure* (objects, facts, rules, quotations), and, while the answer is still open, it proposes individual steps. The symbolic engine handles the *formal* side: it turns the structure into a theory, closes it under inference rules, matches it against the question, and certifies what actually follows.

The central design principle is short: *the language model proposes, the engine decides*. The model never owns truth. Every statement it produces is deterministically classified as `derivable`, `cited` (supported by a verbatim quotation), `hypothesis` (knowledge absent from the text, tagged), or `rejected`. The strength of an answer never exceeds the strength of its premises: a derivation from quotations is `proven`; a derivation resting on assumptions is `proven_under(H)`.

This separation is not an engineering compromise. It reflects the distinction between an *unformalizable source* (doxa) and formal *logos*: the language model here is the bearer of doxa, which logos consults for proposals but never for truth, and the proposal protocol, deterministic

*merfill@yandex.ru

classification, journal, and text-grounding checks form the interface between them [2]. A full account of the operator family is the subject of a separate paper; here we focus on the system that implements it and on its empirical evaluation.

Contributions.

- A propose/decide architecture with two separated participants and a machine-checkable proof trace.
- A proposal protocol in which every statement is classified deterministically, and the strength of an answer (`proven`, `proven_under(H)`, `not_proven`) never exceeds the strength of its premises.
- A family of decidable formalisms under a *declared-fragment protocol*, with honest abstention `out_of_fragment` instead of a silent downgrade to a weaker logic.
- A staged-evaluation methodology for *composite* benchmarks that separates the coverage ceiling (formalism) from the extraction ceiling (model) via a gold-fed mode.
- A common-input comparison with two open-source translator-plus-solver systems on the same rows and the same model, which separates the translation interface from the symbolic reasoning and quantifies the model-regime effect.
- A demonstration prototype on standard collections with a measurable soundness invariant: no confidently wrong conclusions.

Scope (explicit). This is a report on a demonstration prototype, not a claim to state-of-the-art performance. We evaluate *fixed subsamples* of standard collections, explain the budget that prevents full test-set runs, and report confidence intervals. Full coverage of a collection is unattainable by construction for sets with mixed constructions, such as FOLIO (Section 6).

Structure. Section 2 describes the architecture and the proposal protocol; Section 3 gives worked examples per stage; Section 4 lists the formalisms and decision procedures; Section 5 presents the evaluation methodology; Section 6 reports results; Section 7 compares against Logic-LM and LINC on identical inputs; Section 8 discusses limitations; Section 9 covers related work.

2 System overview

Ankyra is not a monolithic solver but a system that coordinates formalizations and decision procedures. For each problem the model produces a formalization in some logic, and a sound procedure for that logic decides it. Different classes of problems require different procedures, each with its own internal representation; the implemented logics are described in Section 4.

2.1 Four phases

A problem passes through four phases. Only one of them — inference — belongs to the engine; the other two consult the model.

- **Phase 0 — decomposition.** The text is split into a descriptive part and a question, and from that point they are treated as separate objects. Two calls to the language model follow in a fixed order: first, a structural decomposition of the descriptive part (objects, facts, conditional statements, alternatives and groupings, negations and modalities, a short verbatim quotation per element); then an expression of the question over the canonical vocabulary already established by the theory. Between and after these calls, a deterministic

builder unrolls and enriches the structure into a *theory* and a *query*. A strict rule applies: *the question is never asserted as a fact*.

- **Phase 1 — decide.** The symbolic engine derives all consequences of the theory to a fixed point and checks whether the goal follows. Every derived fact carries a history: which facts and which rule produced it. The engine adds no knowledge of its own: it is sound and complete *relative to* the formalization it receives.
- **Phase 2 — guided cycle.** If the constructed theory does not yet settle the question, the system does not stop at a bare “unknown”: it reports exactly where the difficulty lies, and the model gets a bounded opportunity to help by proposing one step per iteration (Section 2.3).
- **Phase 3 — explanation.** The finished proof trace is assembled *automatically* from the proof history, not by the model. The model may later paraphrase the finished trace in prose, but it may not introduce new facts.

The key boundary runs between Phases 0 and 1: everything the model formulates must either rest on a verbatim quotation or be explicitly tagged as an assumption. Nothing enters the theory without such grounding.

2.2 The proposal protocol

Every statement the model wishes to add is classified deterministically. The classification is not a subjective judgment: it depends only on whether the statement already follows, is grounded in the text, or is an assumption.

Category	Condition	Engine action
derivable	already follows from the current theory	accepted as narration only
cited	new statement with a verbatim quotation from the text	added to the axioms
hypothesis	knowledge absent from the text	recorded in the journal, tagged <i>H</i>
rejected	fails a schema, safety, or quotation check	dropped with a reason code

It is the distinction between `cited` and `hypothesis` that makes answers honest. A derivation using only quoted facts is `proven`. A derivation resting on assumptions is `proven_under(H)`, and the answer names those assumptions. If nothing is established, the answer is `not_proven`. The strength of an answer therefore never exceeds the strength of its premises, and an assumption cannot pass itself off as a fact.

2.3 The guided cycle and the journal

The inner cycle (Phase 2) is fully deterministic; the only call to the model is a *proposal*. The cycle alternates a hint, a free proposal, and a deterministic classification, and accepts at most one proposal per iteration — so each iteration leaves a checkable trace. An iteration proceeds as follows:

1. **Verify.** The engine closes the current theory and checks the goal, producing a verdict and structured gaps (for example `target_unmatched`: when the goal predicate appears nowhere).

2. **Termination check.** If the verdict is already decisive — **supported**, **refuted**, **contradiction**, or **insufficient** — the cycle stops.
3. **Hint.** The model receives the problem structure, the admissible predicates and objects of the theory, a summary of what has been derived, the current gaps, the goal, and the kinds of proposals allowed.
4. **Proposal.** The model returns a short narrative and exactly one typed proposal: reformulate the question in the theory’s vocabulary, add a rule, assert a quoted fact, or select the next subgoal.
5. **Classification and application.** The engine checks the proposal against the schema, safety rules, and the text, assigns it to one of the four categories above, and either applies or drops it. Application is monotone: the theory only grows.
6. **Record.** The iteration is written to the journal, and the inner cycle restarts.

The cycle stops on a decisive verdict, on **no_progress** (two consecutive iterations without change), on exhausting the iteration budget, or when assumptions are disallowed. None of these outcomes is a correctness violation: each is an honest stopping point.

Monotone base, moving answer. The *base* — axioms, rules, hypotheses — only grows: premises are never deleted, the goal is never weakened. The *answer*, however, may change as new information arrives. Such changes are not hidden: each is recorded as a **Revision** with the source iteration and the assumptions involved, and the revisions are kept alongside the explanation. The system thus remains checkable under non-monotonic reasoning without ever changing its base.

2.4 Data model

Everything the system reasons about is assembled from a few simple elements. An *object* (**Object**) is a canonical named entity. A *relation* (**Morphism**) is a single predicate applied to a subject and an object, with a modality and a negation flag; negation is the same predicate marked false, not a separate name. A *rule* (**Rule**) is a conditional statement: a conjunction of conditions entails a consequence, marked as an implication or an exception and carrying a source (a quotation or a hypothesis identifier). A *theory* (**Theory**) collects objects, asserted relations, and rules, while a *query* (**Query**) stores the conditions the question asserts (Γ), the goal (ϕ), and the answer type. Inside the engine, a *fact* (**Fact**) is the ground atom together with its proof history.

2.5 A principled limitation: deterministic code does not parse language

Deterministic code never interprets natural language. Normalizing identifiers and closed enumerations that the model has *already* extracted (predicate and object identifiers, modality, **relation_kind**, field keys) is permitted: it operates on the structured result, not on the source text. Heuristics over the source text that rely on words, morphology, characteristic phrases, or hand-made term lists are not permitted: they do not generalize across domains, and semantics are set by the extraction instruction. The only operations over the source text are structural checks: exact quotation occurrence, coverage, and length.

3 Worked examples per stage

We give examples for each stage — from pure deduction and abduction through defaults, negation, disjunction, a composite FOLIO case, finite-domain constraints, and exact arithmetic.

They show how the proposal protocol, answer strength, and choice of decision procedure work on concrete inputs.

3.1 Pure deduction

Text: *“It is raining. If it is raining, the ground is wet. Is the ground wet?”*

Phase 0 splits the text into a descriptive part and a question. The descriptive part yields one fact and one rule; the question becomes the goal:

Element	Content	Source
Fact	<code>raining()</code>	cited (“It is raining.”)
Rule R_1	<code>raining() $\Rightarrow$ is_wet(ground)</code>	cited (“If it is raining, the ground is wet.”)
Goal ϕ	<code>is_wet(ground)</code> , type <code>yes_no</code>	question (not a fact)

Phase 1 closes the theory: rule R_1 fires on the fact and derives `is_wet(ground)`, carrying the premise and the firing rule in its history. Verification gives the verdict **supported**, the answer **yes**, and strength **proven**; the cycle makes no call to the model. Phase 3 assembles the trace automatically:

1. `raining()` — axiom, cited;
2. `is_wet(ground)` — rule R_1 , from step 1.

3.2 Abduction and answer strength

Text: *“The vehicle has an engine, four wheels, 150 horsepower and four doors. What kind of vehicle is it?”*

Phase 0 extracts only the observed facts — `has_engine`, `wheel_count`, `power`, `door_count`. The text contains no classification rules, so the theory has none either. The goal is open: `is_a(X, ?c)`. Phase 1 derives nothing for this predicate and reports the gap `target_unmatched: is_a`, after which the guided cycle starts.

In the first iteration the model proposes classification rules, for example: “engine $\Rightarrow$ motor vehicle”, “motor vehicle and four wheels $\Rightarrow$ car”. There are no quotations under them, so the classification tags them as **hypothesis**, and the rules are entered in the journal with tags $H_1, H_2, \dots$ (Section 2.3). The theory grows monotonically, closure derives `is_a(X, car)` and `is_a(X, sedan)`, and the goal variable is bound along the `is_a` chain rather than guessed. Verification gives **supported**, but the derivation rests on assumptions, so the answer strength is **proven_under**($H_1, H_2, \dots$), and the answer names those assumptions.

Running the same problem with `ANKYRA_ALLOW_HYPOTHESES=false` rejects the same rules, and the system answers honestly **not_proven**. The difference between **proven** and **proven_under** in these two examples is not cosmetic: it reflects exactly what the answer rests on.

3.3 Defaults and specificity

Text: *“Birds can fly. Penguins are birds, but penguins cannot fly. Tweety is a penguin. Can Tweety fly?”*

Phase 0 gives the facts `is_a(tweety, penguin)` and `is_a(penguin, bird)`, together with two default rules: `is_a(?x, bird)  $\Rightarrow$  fly(?x)` and `is_a(?x, penguin)  $\Rightarrow$   $\neg$ fly(?x)`. Under plain monotone closure both rules would fire and produce a contradiction. With the defaults layer enabled, the penguin rule is strictly more specific — it rests on the edge `is_a(penguin, bird)` — and wins: the verdict is **refuted**, the answer is “no”, and the flag `defeasible=true`.

The symmetric example — “birds do not fly, penguins fly” — gives **supported** and the answer “yes”: specificity does not depend on polarity. In the classic example “Nixon is a Quaker and a Republican” with rules `quaker  $\Rightarrow$  pacifist` and `republican  $\Rightarrow$   $\neg$ pacifist`, specificity is undefined — both rules are equally specific — and the system honestly answers **unsupported** with the gap `undecided_conflict`, rather than guessing a winner. Source: `evals/defeasible_synthetic`, 8/8.

3.4 Negation and disjointness (L1)

Text: *“No real number is imaginary. a is a real number. Is a imaginary?”*

The statement “no real number is imaginary” is formalized not as a general rule but as a disjointness constraint `disjoint(real_number, imaginary)`. Together with the fact `is_a(a, real_number)`, it yields `$\neg$ is_a(a, imaginary)` and hence refutes the goal: verdict **refuted**, answer “no”. It is essential that the refutation rests on an explicit disjointness rather than on a silent closed-world assumption: under the open world, the system does not declare the goal false merely because it failed to derive it. Source: `evals/l1_synthetic`, identifier `disjoint-refute-01`; gate 40/40.

3.5 Disjunction and case split (L2)

Theory: fact `is_a(rex, prim)`; rule `is_a(?x, prim)  $\Rightarrow$  is_a(?x, a)  $\vee$  is_a(?x, b)`; rules `is_a(?x, a)  $\Rightarrow$  is_a(?x, c)` and `is_a(?x, b)  $\Rightarrow$  is_a(?x, c)`; goal `is_a(rex, c)`. The Horn procedure cannot handle a disjunctive head, so the clausal L2 procedure is enabled: it splits into cases over the two disjuncts and derives `c` in each, after which the verdict is **supported**. With L2 disabled, the same input yields an honest `out_of_fragment`, not a silent downgrade to a weaker logic. Source: `evals/l2_synthetic`, identifier `head-case-split-01`; gate 65/65.

3.6 A composite FOLIO case

The FOLIO gold formalization mixes constructions from several stages, so the collection is evaluated by intra-fragment slices. In row `folio-validation-0010` (“wild turkey”), the premises contain a disjunction of six subspecies and the fact `WildTurkey(tom)`. Conditional negations together with this fact yield `$\neg$ Eastern(tom)`, `$\neg$ Osceola(tom)`, and the negation of the Goulds/Merriams/Riogrande disjunction; the goal `Eastern(tom)`, labeled **False**, is therefore refuted. The clausal procedure gives **refuted** (“no”), matching the label.

Row `folio-validation-0027` (“Marvin”) is different: the goal is labeled **False**, but the annotated premises entail neither the goal nor its negation, and the system honestly abstains (**unknown**) rather than issuing an unconfirmed “no”. On the negation slice (13 rows) the gold-fed mode gives 12/13 with zero confidently wrong conclusions; on the L2 tier-a slice (45 rows) coverage is 44/45 and gold-fed is 42/44. Sources: `evals/data/folio_l2_tier_a.jsonl`, `folio_negation_tier_a.jsonl`.

3.7 Finite-domain constraints (L3)

An AR-LSAT problem: five instrumental pieces run on a compact disc in some order; conditions of order, adjacency, and grouping are given, and the question asks which of the proposed sequences is admissible. The formalization is finite-domain: a domain of positions 0..4, five variables with an `all_different` constraint, ordering and composite constraints. A `not_violate` question is checked by enumeration, and “must be” and “could be” questions are decided as truth in all or at least one admissible placement. On hand-encoded real games the gold-fed mode gives 27/27, including 5/5 for `not_violate` questions; the live run on the eval split gives 23/30. Sources: `evals/data/ar_lsats_dev.jsonl`, `evals/data/ar_lsats_gold.jsonl`.

3.8 Exact arithmetic (L4)

A GSM8K problem: “Henry made two stops during a 60-mile trip; the first was after 20 miles, the second was 15 miles before the end. How many miles did he drive between the stops?” Phase 0 builds a directed graph of quantities: `total=60`, `first=20`, `before_end=15`, and the target quantity `between` is given by an equation. The linear system is solved in exact rational arithmetic: $60 - 20 - 15 = 25$. An arithmetic error is impossible here by construction: a wrong number can only be a *modeling* error, never a counting one. Source: `evals/data/gsm8k_gold.jsonl`, identifier `gsm8k-gold-0028`; gold-fed 8/8.

4 Formalisms and decision procedures

Ankyra is not a “Horn-clause solver” but a system that coordinates formalizations. The end goal is to cover the largest practically important class of reasoning for which a decidable decision procedure exists: the model produces a formalization in a logic L , and a sound procedure for L decides it. Coverage is therefore measured in named formalisms, not in an undefined notion of “reasoning”.

4.1 The stage protocol

Each stage is described by the same six items: the *formalism* L and the answer semantics; the *decision procedure*, sound for L (and, where claimed, complete); the *engine change*; the *extraction change* — what Phases 0 and 2 must be able to express for the model to propose in L ; the *staged gate on a reference collection* with a small fixed subsample and hard invariants (no confidently wrong conclusions, every determinate answer **proven**, accuracy above threshold, every remainder categorized); and a *switch*, off by default, so that the semantics of the reference collection does not leak into the engine.

Stage	Formalism	Procedure
L0	definite Horn clauses, <code>is_a</code> , negation as complementary pairs (open world)	semi-naive forward chaining to a fixed point
L1	stratified negation / negation as failure (NAF), declared closed world	stratified closure, disjointness constraints
L2	positive first-order logic: disjunction, $\exists/\forall$, proof by cases	bounded resolution (grounding over a finite domain)
L3	finite-domain constraints (CSP/SAT)	search over finite value domains
L4	arithmetic terms and equations	exact rational evaluation / linear elimination
D	defaults with specificity via <code>is_a</code>	defeasible layer (specificity over the strict core)

L3 and L4 are separate engines within the same system, each with its own internal representation: finite-domain constraints are not logical derivability but constraint satisfaction, and an arithmetic problem yields a number, not a consequence.

4.2 The declared fragment

Since the procedures differ, the system must decide which one runs — and do so without ever guessing. The *declared-fragment protocol* provides the answer. The system analyzes the *constructed* theory and query and derives the set of required features: is there a disjunctive head,

an existential quantifier, a composite goal, a negation, a built-in comparison, an equality? This is the *fragment* — an objective property of the formalization, not an intention. Separately, a run *declares* its semantics: which procedures are enabled (`ANKYRA_*` switches) and whether the query is read under an open or closed world. The routing decision combines the two and checks them against each other.

If the fragment fits within the enabled capabilities, the corresponding procedure is chosen. If not — the structure needs a logic that the run did not enable, or a combination the chosen procedure cannot honor — the result is the named gap `out_of_fragment`, not a silent downgrade to a weaker procedure.

The extension point is the **Inference** protocol: the semantics (derivability, verdict shape, closure against which a proposal is classified) become swappable, while procedure selection and mechanisms stay in place. There are currently two implementations — Horn and clausal; L3 and L4 are plugged in as separate engines.

4.3 What the engines share

Every engine is sound *relative to* the formalization it receives, and every engine abstains honestly instead of guessing. The symbolic components are complete relative to the formalization (forward chaining reaches a fixed point), but completeness of the *translation* is not claimed: the model can always miss a premise in the text. The distinction between a wrong answer and a missing one is therefore preserved throughout.

5 Evaluation methodology

Metrics. For each collection we report *class accuracy* (three-class or multiple-choice) with Wilson 95% confidence intervals. The staged gate also requires a *soundness invariant*: every determinate answer **proven** and no confidently wrong conclusions.

Two ceilings. For a composite reference collection we separate *coverage* (the share of gold formulas the implemented logic can express and decide; the remainder is an honest `out_of_fragment`) and *extraction* (the share of covered rows the language model translates correctly). The *gold-fed mode* feeds the engine annotated formalizations instead of model-extracted ones, separating the accuracy of the method from the accuracy of translation.

Budget and reproducibility. There is no free access to the language model; runs that call it are paid. Each collection therefore enters as a small fixed subsample, runs once, and is repeated only on a discrepancy. All checks without a language model (synthetic sets, gold-fed diagnostics) are reproducible from a single repository.

The evaluation ladder. Results are obtained on a three-rung ladder. The first rung is synthetic sets without a language model, directly exercising the semantics of each formalism, with no extraction and no provider variance: this is the clean engine signal. The second rung is a surface-simple real collection (for example ProntoQA-OOD for L2), adding real text on top of clean notation. The third rung is a deliberately extraction-heavy slice of a real collection (FO-LIO). Across the ladder, the gold-fed mode — feeding the engine hand-written formulas instead of extracted ones — separates two distinct ceilings: *coverage* (how many problems the implemented logic can express at all; going beyond it is an honest `out_of_fragment`, costing completeness but not soundness) and *extraction* (how many covered problems the model translates correctly). The separation shows where improvement must come from — the engine or the interface.

6 Results

We report two results: class accuracy on covered slices and the soundness invariant. Deterministic checks without a language model come first, followed by live runs on external collections. On no reported slice is there a single confidently wrong conclusion, and every determinate answer is tagged **proven**.

6.1 Synthetic checks without a language model

These checks are deterministic and cover every construction of every formalism (including negative controls); the language model is not in the loop.

Check	Result	Confidently wrong
Procedure selection (<code>evals.routing_synthetic</code>)	19/19	0
L1 negation (<code>evals.l1_synthetic</code>)	40/40	0
L2 clausal + equality (<code>evals.l2_synthetic</code>)	65/65	0
L3 finite-domain CSP (<code>evals.l3_synthetic</code>)	31/31	0
L4 arithmetic (<code>evals.l4_synthetic</code>)	37/37	0
Defaults (<code>evals.defeasible_synthetic</code>)	8/8	0

6.2 External collections

Table 1 summarizes the live and fixed results. The confidently-wrong column is the soundness invariant (0 in every row).

Stage	Collection / slice	Result	95% CI
L0	ProofWriter, tier D (open world)	297/299 (99.3%)	[97.6; 99.8]
L1	ProntoQA, tier a	48/48 (100%)	[92.6; 100]
L1	ProntoQA, tier b	160/160 (100%)	[97.7; 100]
L2	ProntoQA-OOD, tier a	41/42 (97.6%)	[87.7; 99.6]
L2	FOLIO, negation slice (text-fed)	11/13 (84.6%)	[57.8; 95.7]
L2	FOLIO L2, tier a (text-fed, covered)	29/44 (65.9%)	[51.1; 78.1]
L3	AR-LSAT, eval	23/30 (76.7%)	[59.1; 88.2]
L4	GSM8K, eval	38/40 (95.0%)	[83.5; 98.6]

Table 1: Results on fixed subsamples. The denominators are small samples, not full test sets; see Section 8.

ProofWriter (L0). This is the base gate for the definite Horn layer. On the small strict tier-A slice it is 45/45; on the extended tier D it is 297/299 (99.3%) over evaluated records, with no confidently wrong conclusion and every determinate answer **proven**. One of the 300 selected records is excluded from the denominator: its question could not be extracted (provider variance), and a targeted rerun answered it **proven**. The two remaining misses are honest **unknown** answers under an extraction error, not wrong conclusions. The full collection (6,368 theories, roughly 11 hours) was not run for budget reasons.

ProntoQA (L1) and ProntoQA-OOD (L2). These are gates on real text over clean notation. ProntoQA checks explicit negation: tier a is 48/48 and tier b is 160/160, all answers **proven**; the one-sided 95% Clopper–Pearson lower bound on per-problem accuracy for tier b is 98.1%. ProntoQA-OOD checks compositional reasoning (disjunction, case split, quantifiers): tier a is 41/42 (97.6%), with no confidently wrong conclusion. After these runs, ProntoQA is considered closed.

FOLIO (L1/L2, composite case). The FOLIO first-order annotation spans several formalisms at once, so it is evaluated by intra-fragment slices rather than by a single score:

- The negation slice (13 rows) turned out to be *fragment-bound*: on the Horn path, gold-fed and text-fed modes coincide (7/13), and the remainder is reductio and contraposition; the slice is therefore scored by the clausal L2 procedure: gold-fed 12/13, text-fed 11/13, with no confidently wrong conclusion. The four **Uncertain** answers remain honest **unknown**; a global closed world is not applied, since it would wrongly refute them.
- The L2 tier-a slice (45 rows): coverage is 44/45 (one broken annotation), i.e. the implemented logic expresses almost the whole slice; on the covered rows, gold-fed gives 42/44 against 29/44 under text-fed.

The gap between gold-fed (method) and text-fed (extraction) localizes the remaining cost in extraction, not in the engine: the coverage ceiling on the fixed 45-row slice is in effect reached. Across the whole 204-row validation split, the two fixed gates cover $13 + 93 = 106$ rows (a lower bound on the expressible share); of the 93 rows of the L2 fragment, only 45 are fixed and run, while 48 are left unrun for budget reasons. The remaining rows — at most 98 — contain constructions outside any implemented stage (exclusive or, equivalence, multi-variable quantifiers, equality, function terms, axiom schemas) and are reported as `out_of_fragment`.

AR-LSAT (L3). Finite-domain constraints are checked by a separate engine. On hand-encoded real games the gold-fed mode gives 27/27; the live run on the eval split is 23/30 (76.7%) with no `grounded_mismatch`. The seven misses are honest abstentions, not confidently wrong answers.

GSM8K (L4). Exact rational arithmetic makes arithmetic errors impossible by construction; a wrong number here is always a *modeling* error, never a counting one. Synthetic checks are 37/37, the hand-encoded gold is 8/8, and the live run gives dev 12/12 and eval 38/40. The two wrong rows in the eval split are annotated gold errors: one dataset error, one ambiguity.

7 Comparison with Logic-LM and LINC

The formalisms and gates above are this paper’s contribution. A natural question is how the same problems are handled by the two closest open-source neuro-symbolic systems, Logic-LM [3] and LINC [4]. Both are translator-plus-solver pipelines: a language model writes a formalization and a symbolic prover decides it. Rather than contrast them qualitatively (as in Section 9), we report a *common-input* comparison — the same problem rows and the same language model for all three systems — and then analyse what separates them.

7.1 Protocol

Inputs and model. Every system receives the exact Ankyra sample rows and the same model the engine uses at run time: a small, inexpensive instruction model (DeepSeek V4 Flash) behind an OpenAI-compatible endpoint. The external systems were not reimplemented. Their own prompt, symbolic back-end and scoring are driven unchanged by a thin adapter; because both were written against the legacy `openai<1` client, that client is redirected to the engine’s provider, forced to the engine’s model, and configured to disable the model’s explicit thinking trace exactly as the engine does. Temperature-0 calls are cached, so the runs are reproducible at no further cost.

Decoding. Each system runs its own protocol, because that is the protocol its published accuracy refers to. Sampling here means running the language model several times on the same

input to obtain N candidate outputs. It is used to estimate the stability of the generation and to raise the chance that at least one candidate passes parsing.

System	Decoding	Why
Ankyra	temperature 0.1, one sample, structured (tool/JSON) extraction	the engine’s design; it never samples
Logic-LM	temperature 0, one sample, plus self-refinement	its published pipeline
LINC	8-shot, 10 samples, temperature 0.8, majority vote	its published pipeline

A matched-decoding ablation (all three greedy, one sample) is given in Section 7.6, so that the protocol itself is not silently doing the work.

Metrics. *Solver accuracy* is the share of rows with a correct answer. Both a failed program execution and an abstention count as wrong; this is the strictest of our metrics, and we use it throughout. *Coverage* is the share of rows whose program executed. *Executable accuracy* is accuracy over the rows where the program actually ran. *Confidently wrong answers* are executed programs with an incorrect answer: the external systems commit to such an answer, whereas Ankyra is sound relative to its formalization and produces no such answers.

Scope. This is not a reproduction of the two papers. They used GPT-3.5/GPT-4 and their own splits; fixing the rows and the model makes the systems comparable but changes every absolute number. The object of interest is the *relative* behaviour of the three pipelines under identical input.

7.2 Results

Solver accuracy and coverage are given in Tables 2 and 3; Ankyra’s numbers are the gates of Section 6.

Stage	Collection (n)	Ankyra	Logic-LM	+refine	LINC
L0	ProofWriter tier A (45)	100.0	8.9	—	86.7
L0	ProofWriter tier D (300)	99.0	6.7	—	28.3*
L1	ProntoQA tier a (48)	100.0	41.7	—	—
L2	FOLIO L2+negation (58)	69.0	6.9	39.7	74.1
L3	AR-LSAT eval (30)	76.7	53.3	63.3	—

Table 2: Common-input comparison: solver accuracy in percent of all rows (non-executed programs and abstentions counted wrong). LINC runs its native protocol except *ProofWriter tier D, which was run greedily (its one-sample value on tier A is 22.2%). “—” means the system does not support the collection. Logic-LM’s +refine column applies its self-refinement where the released code implements it (FOLIO, AR-LSAT).

Collection (n)	Ankyra	Logic-LM	+refine	LINC
ProofWriter tier A (45)	100	13.3	—	100
ProofWriter tier D (300)	100	7.7	—	32.0*
ProntoQA tier a (48)	100	41.7	—	—
FOLIO L2+negation (58)	98.3	6.9	55.2	98.3
AR-LSAT eval (30)	100	60.0	70.0	—

Table 3: Coverage: percent of rows whose symbolic back-end executed.

The counts of confidently wrong answers are: Ankyra 0 everywhere; Logic-LM 2 (tier A), 3 (tier D), 0 (ProntoQA), $0 \rightarrow 9$ with self-refinement (FOLIO), 2 (AR-LSAT); LINC 6 (tier A), 11* (tier D), 14 (FOLIO). Ankyra’s zero is structural (Section 7.4); an external zero only means the executed subset happened to be correct.

A first impression is misleading, and two facts confirm this. First, LINC is competitive rather than broken: on FOLIO it reaches 74.1%, *above* Ankyra’s 69.0%. Second, Logic-LM’s low figure is real for this model: even with two rounds of its own self-refinement it reaches only 39.7% on FOLIO and 63.3% on AR-LSAT, and it offers no self-refinement for its PyKE datasets.

7.3 Translation, not reasoning

The symbolic back-ends are not the bottleneck. Feeding them hand-written formalizations of the same 58 FOLIO rows, with no model in the loop:

System	Back-end	Gold-fed solver acc.	Gold-fed exe. acc.
LINC	NLTK/Prover9	54/58 (93.1%)	54/57 (94.7%)
Logic-LM	Prover9	47/58 (81.0%)	47/50 (94.0%)

Running Logic-LM’s solver on its own committed GPT-4 programs likewise reproduces paper-level behaviour offline (executable accuracy: FOLIO 80.4%, ProofWriter 79.6%, ProntoQA 83.2%, AR-LSAT 60.0%). The provers work; the interface fails: Logic-LM and LINC ask the model to write a program as text and then parse that text with a fragile parser, whereas Ankyra asks for structured JSON through tool calling and receives it without parsing. Re-executing the model’s own saved programs offline shows exactly where the failure occurs:

Collection (n)	Executed	Exec. error	Parse fail.	Bullets	bare x
ProofWriter tier D (300)	23	214	63	276	179
FOLIO (58)	4	0	54	50	55
ProntoQA tier a (48)	20	21	7	28	—
AR-LSAT (30)	18	12	0	0	0

The dominant failures are surface syntactic details the model did not reproduce: Markdown bullet lists, bold text, and bare x where Logic-LM’s grammar requires $\$x$. AR-LSAT uses a z3-expression grammar that the model mostly reproduces; its twelve failures are defects in Logic-LM’s own z3 code translator (raising the 1 s execution timeout to 10 s changes nothing).

7.4 The interface is the difference

The three systems require different output from the model. Ankyra extracts into a JSON schema through tool/function calling, and a deterministic builder turns that structure into the theory; format compliance is enforced by the API contract, not by the prompt, so the model only has to get the *content* right. Logic-LM requires a program in a hand-written grammar (**Predicates:/Facts:/Rules:/Query:**, **:::**, $\$x$) read by a parser sensitive to syntax. LINC requires a **TEXT:/FOL:** scaffold and executes the extracted formulas; the scaffold is simple enough that a few of ten samples usually fit, which is why its sampling matters so much (Section 7.6). The central finding is that with a small, inexpensive model the binding constraint is the *translation interface*, not the reasoning: Ankyra is nearly invariant to the model’s formatting habits, while the free-text pipelines can break easily.

7.5 Model sensitivity

To understand how much of the result comes from the LLM itself and how much from the pipeline, we compared LINC’s own ProofWriter test set under LINC’s own code and protocol on a fixed

60-row subset (every sixth row). Only the DeepSeek column is our run. The GPT-4 and GPT-3.5 values are *not* runs performed here: they are recomputed from the generations the LINC authors committed to their repository, whose configuration (8-shot, ten samples, temperature 0.8) matches the protocol we used and whose reference labels align row-for-row with ours. The comparison is therefore of model regimes over the same task and configuration, not of three runs performed in this study.

Model (same task, code and configuration)	Accuracy
GPT-4 (LINC’s committed generations)	100.0%
GPT-3.5-turbo-16k (LINC’s committed generations)	96.7%
DeepSeek V4 Flash (our run)	85.0%

The model alone accounts for about twelve points, and LINC’s published 96–98% on the full test is consistent with the GPT rows. The external systems’ published numbers therefore rest on GPT-3.5/GPT-4; Ankyra’s contribution here is that it holds up under the weaker model.

7.6 Decoding ablations

LINC: greedy versus native. On the same rows and tasks, moving LINC to its native protocol raises FOLIO from 34.5% to 74.1% and ProofWriter tier A from 22.2% to 86.7%. LINC’s robustness lives substantially in its sampling/majority step; reporting only the greedy value would be misleading.

Logic-LM: best-of- N . Logic-LM does not sample. As a control, five samples per row at temperature 0.8 on ProofWriter tier A yielded *zero* executable programs out of 225 (the greedy run produced 6 of 45). Higher temperature worsens its grammar compliance rather than rescuing it, so sampling does not explain the two systems’ different behaviour.

Logic-LM: self-refinement. Its own refinement, implemented only for Prover9/FOLIO and z3/AR-LSAT, roughly triples FOLIO coverage (6.9% $\rightarrow$ 55.2%), raises FOLIO solver accuracy from 6.9% to 39.7% and AR-LSAT from 53.3% to 63.3%, and introduces confidently wrong answers on FOLIO (0 $\rightarrow$ 9).

7.7 What is not run, and why

The comparison is bounded by the code of the systems under evaluation, not by arbitrary choice. LINC’s task registry contains only FOLIO and ProofWriter, so it cannot be run on ProntoQA, AR-LSAT or GSM8K; Logic-LM has no GSM8K path. Logic-LM’s self-refinement exists only for Prover9 and z3 — its prompt directory contains `self-correct-AR-LSAT.txt` and `self-correct-FOLIO.txt` and nothing else, and its executor map lists only those two back-ends — so there is no self-refinement to run for its PyKE datasets (ProofWriter, ProntoQA). Per-prover capability was still measured, deliberately and without the model, by the gold-fed runs above. Stage L4 (arithmetic) and ProntoQA-OOD have no counterpart in either system and remain out of scope.

7.8 Caveats

Ankyra’s figures are taken from the committed gates and live traces, but were not re-run on the same day as the external systems; one ProofWriter tier-A trace is an empty-question extraction failure (provider variance), so the saved traces show 44/45 against the accepted gate 45/45. LINC tier D is greedy rather than native, so it is a lower bound. Provider non-determinism is real even at temperature 0; single runs are reported, as everywhere in this paper. Finally, the

comparison is end-to-end — prompt, decoding, parser and solver together — which is exactly what is needed to see how well a method turns a small model into a sound answer.

8 Limitations

Sample size and statistics. Every number on a real collection rests on a small sample ($N = 13, 30, 40, 42, 44, 300$), so the confidence intervals are wide (Table 1); on the small slices they overlap with plausible baselines and do not support a state-of-the-art claim. Detecting a difference of about five percentage points at $p \approx 0.95$ with power 0.8 would require on the order of 400 examples per group.

Single runs and non-determinism. The live path calls a language model whose provider is non-deterministic even at zero temperature. Runs are performed once (for budget reasons) and repeated only on a discrepancy, so we do not report a variance estimate and do not perform significance tests.

Subsamples, not full test sets. We evaluate fixed subsamples: ProntoQA-OOD tier a (42 of 5,800), AR-LSAT eval (30 of ~ 230), GSM8K eval (40 of $\sim 1,319$), ProofWriter tier D (300 of 6,368 theories). Full collections were not run because there is no free access to the language model and the runs are paid for by the author out of pocket; this is a deliberate boundary of the study, not a hidden failure.

Soundness is relative to the formalization. The engine is sound *relative to* the theory it is handed; the translation from natural language into that theory is done by the language model. An extraction error can therefore produce an *abstention* or, in principle, a **proven** answer about a wrong formalization. We measure the absence of confidently wrong conclusions (zero on every reported slice) and reduce extraction risks through verbatim quotation checks, guards against citing conditional statements, guards against refuting on the basis of hypotheses, and domain-declaration control. This is not an end-to-end soundness proof relative to the real world.

No single aggregate for a composite collection. For FOLIO, full coverage of the collection is unattainable by construction: equality, function terms, exclusive or, equivalence, multi-variable quantifiers, and axiom schemas lie outside any implemented stage, and full first-order derivability is only semi-decidable. We therefore report stratified slices and an explicit `out_of_fragment` counter rather than a whole-collection score.

Comparison with published results. Section 7 does run a common-input comparison, but not against the *published* numbers: published values are obtained on different collections, splits, protocols, and models, so the reported accuracies remain not directly comparable across papers. The comparison is with the released systems re-run on identical inputs and one model; the contribution remains an argument about soundness, checkability and robustness to the translation model, with accuracy as supporting evidence.

Model and provider variability. The numbers are tied to the extraction provider at run time; a different or updated model will change the text-fed columns. The gold-fed columns and the synthetic checks do not depend on the model.

Defaults. Only the synthetic gate is reported; a real defeasible/natural-language-inference (NLI) collection remains future work.

9 Related work

Map of the field. A 2026 systematic survey [5] (178 works, 2020–2025) identifies four stable patterns of neuro-symbolic integration: sequential cascade, symbolic validator, symbolic planner with a neural executor, and differentiable integration. Ankyra is closest to the sequential cascade with a bounded cycle, but it differs in two respects: a proposal protocol under which every statement enters the theory only with a quotation or an assumption tag, and a declared fragment under which a mismatch between the required and available logic yields an honest `out_of_fragment` rather than a silent downgrade to a weaker procedure. The same survey notes that meta-cognitive abilities — monitoring one’s own conclusions, recognizing a lack of knowledge, switching modes — appear in only 5% of the works; `out_of_fragment` and `proven_under(H)` can be seen as a contribution to exactly this niche.

The closest system in spirit is *Autonomous Trustworthy Agents* (ATA) [6]: it also pursues determinism and checkability, but achieves them by *removing* the LLM from the runtime entirely (the LLM builds a knowledge base offline; only the symbolic solver runs online). Ankyra keeps the LLM in the loop, constraining it through the protocol; the two are different points on the same spectrum. The same approach to verifying regulatory documents with an SMT solver is described in [7].

Language-model reasoning without a certificate. Chain-of-thought [8] and self-consistency [9] show that step-by-step text generation improves language-model answers. Such chains, however, are not certified: a step that looks plausible may be wrong, and the answer carries neither a proof history nor a distinction between “proven” and “plausible”. Ankyra addresses exactly this gap: the model produces the reasoning, but a symbolic engine checks and certifies derivability.

Language model and symbolic engine. Translator-plus-solver pipelines (Logic-LM [3], LINC [4]) also hand reasoning over to a symbolic solver, but trust the translation entirely: if the formalization is wrong, so is the conclusion. Here the boundary is stricter: every statement enters the theory only with a verbatim quotation or an explicit assumption tag, answer strength is derived from the premises (`proven` versus `proven_under`), and a mismatch between the required and available logic yields an honest `out_of_fragment` rather than a hidden downgrade to a weaker procedure. Section 7 re-runs both systems on identical inputs and the same model: their provers are sound and capable (gold-fed 81–93% on FOLIO), but as free-text pipelines they are sensitive to a small model’s formatting, while the structured interface of Ankyra is not.

Collections as staged gates. We use standard collections — ProofWriter [10], ProntoQA [11] and ProntoQA-OOD [12], FOLIO [13], AR-LSAT [14], GSM8K [15] — not as a single aggregate score but as staged gates for individual stages, each with its own semantics. For composite collections (FOLIO) we isolate intra-fragment slices and separately report `out_of_fragment`; a single score across a collection with mixed constructions is unattainable by construction.

Non-monotonic reasoning and defaults. Default logic [16] and circumscription [17] formalize the defeasibility of conclusions; Ankyra implements a defaults layer with specificity via `is_a` and honestly reports an undecided conflict rather than guessing a winner. The philosophical framing (doxa/logos) and the operator family of interaction are described in a companion paper [2]; here we focus on the system that implements this interface.

Proof history and checkability. The reliance on an automatically assembled proof history aligns Ankyra with work on checkable proof. The explanation here is a presentation of the

derivation, not a second opinion about it: the model may not add facts to it, and every step names the rule that fired, the bindings used, and the supporting quotation or hypothesis.

10 Conclusion

We have presented a neuro-symbolic system in which a language model proposes and a deterministic engine decides, spanning a family of decidable formalisms under a declared-fragment protocol. The contribution is not an individual solver but an architecture and a discipline: a proposal protocol in which no statement enters the theory without a quotation or an assumption tag; answer strength that never exceeds the strength of its premises; a declared fragment and honest `out_of_fragment` instead of a silent downgrade to a weaker logic; and a staged-evaluation methodology for composite collections that separates the method’s coverage from the extraction ceiling.

On fixed slices of standard collections the system achieves high accuracy without a single confidently wrong conclusion, and honestly abstains where its logic cannot express the problem. A common-input comparison with Logic-LM and LINC shows that their symbolic reasoning is sound but their free-text translation interface is the wall under a small model, whereas Ankyra’s structured extraction is robust to it; with a stronger model and its native sampling, LINC is competitive. The evaluation is a demonstration prototype, not state of the art; the limitations above define the boundary of what these results support. Natural directions for further work are a real defeasible/NLI collection, finite function terms and multi-variable quantifiers (currently outside the fragment), and runs on full collections as budget becomes available.

A Fragments and switches

The system derives the required fragment from the *constructed* theory and query; the switches declare which procedures a run is allowed to use. Neither is set by the model.

Fragment feature	When it arises	
horn	no non-Horn heads, no existential quantifiers, single goal	
negation	non-empty disjointness constraints or a negative literal	
disjunction	a disjunctive head or a disjunctive ground fact	
existential	non-empty existential theory statements or an existential goal	
compound_goal	goal_mode $\neq$ single (conjunctive or disjunctive goal)	
builtin	a comparison predicate (<code>=</code> , <code>neq</code> , <code><</code> , <code>lte</code> , <code>></code> , <code>gte</code>)	
equality	the reserved predicate <code>eq</code> anywhere	

Switch	Default	Action
ANKYRA_MAX_WAVES	8	maximum reasoning iterations
ANKYRA_ALLOW_HYPOTHESES	true	whether assumptions are allowed
ANKYRA_NEGATION_MODE	open	default world assumption for L1
ANKYRA_LOGIC	off	enable the clausal L2 procedure
ANKYRA_DEFEASIBLE	off	enable the defaults layer
ANKYRA_BUILTINS	false	enable the comparison layer
ANKYRA_CSP	off	enable the finite-domain L3 engine
ANKYRA_ARITH	off	enable the numeric L4 engine
ANKYRA_EXTRACT_SAMPLES	1	number of extraction attempts

B Reproduction

Checks without a language model (synthetic sets and gold-fed diagnostics) are reproducible from the repository without access to the language model and without API payment:

```
uv run python -m evals.routing_synthetic
uv run python -m evals.l1_synthetic
uv run python -m evals.l2_synthetic
uv run python -m evals.l3_synthetic
uv run python -m evals.l4_synthetic
uv run python -m evals.defeasible_synthetic
uv run python -m evals.ar_lsats --gold evals/data/ar_lsats_gold.jsonl
uv run python -m evals.gsm8k --gold evals/data/gsm8k_gold.jsonl
uv run python -m evals.analyze_folio
```

Runs that call the language model (to the paid extraction provider) are performed once and repeated only on a discrepancy:

```
uv run python -m evals.proofwriter --tier d
uv run python -m evals.prontoqa --tier b
uv run python -m evals.prontoqa_ood --tier a
uv run python -m evals.ar_lsats --sample eval --live
uv run python -m evals.gsm8k --sample eval --live
```

The common-input comparison of Section 7 is reproducible from the repository: adapters and drivers live in `comparison/` and the per-row outputs in `comparison/results/`. The offline parts (adapters, gold-fed back-end checks) need no API; the live parts are the paid runs recorded there.

```
PYTHONPATH=$PWD /tmp/opencode/cmp/llm-venv/bin/python -m comparison.run_logicllm \
  --repo /tmp/opencode/cmp/logicllm --action prepare
PYTHONPATH=$PWD /tmp/opencode/cmp/llm-venv/bin/python -m comparison.run_logicllm \
  --repo /tmp/opencode/cmp/logicllm --action all --collection folio_tier_a
PYTHONPATH=$PWD /tmp/opencode/cmp/linc-venv/bin/python -m comparison.run_linc \
  --repo /tmp/opencode/cmp/linc --collection folio_tier_a --nshot 8 \
  --n-samples 10 --temperature 0.8 --jobs 6
```

References

- [1] Ankyra: A domain-agnostic neuro-symbolic reasoning engine. GitHub repository. URL: <https://github.com/merfill/ankyra>
- [2] Lapshin V. Doxa and Logos: Formalizing the Functions of a Non-Formalizable Source in Neuro-Symbolic Systems. (companion paper, in preparation).
- [3] Pan L. et al. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. Findings of EMNLP. 2023.
- [4] Olausson T. X. et al. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. EMNLP. 2023.
- [5] Bin Hakim S., Adil M., Velasquez A., Song H. H. Neuro-symbolic agentic AI: Architectures, integration patterns, applications, open challenges and future research directions // Computer Science Review. 2026. Vol. 60. Art. 100902. DOI: 10.1016/j.cosrev.2026.100902.
- [6] Peer D., Stabinger S. ATA: Autonomous Trustworthy Agents. arXiv:2510.16381.

- [7] Hsia Y.-S., Yu F., Jiang J.-H. R. Neuro-Symbolic Compliance: LLM-driven SMT Verification of Regulatory Documents. arXiv:2601.06181.
- [8] Wei J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903.
- [9] Wang X. et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. arXiv:2203.11171.
- [10] Tafjord O., Mishra B. D., Clark P. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. AI2, arXiv:2012.13048.
- [11] Saparov A., He H. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. ICLR 2023. arXiv:2210.01240.
- [12] Saparov A., He H. Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples. NeurIPS 2023. arXiv:2305.15269.
- [13] Han S. et al. FOLIO: Natural Language Reasoning with First-Order Logic. arXiv:2209.00840, v3 2024.
- [14] Zhong W. et al. AR-LSAT: Investigating Analytical Reasoning of Text. 2021. arXiv:2104.06598.
- [15] Cobbe K. et al. Training Verifiers to Solve Math Word Problems. 2021. arXiv:2110.14168.
- [16] Reiter R. A Logic for Default Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 81–132.
- [17] McCarthy J. Circumscription—A Form of Non-Monotonic Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 27–39.